

LLM AI commander (OPFOR)

This document describes the optional language-model commander for the OPFOR (RED) coalition: what it is allowed to do, how a turn flows through it, the fairness guarantees it makes, how to audit a turn after the fact, how to configure a provider, what a turn costs, and what is deliberately out of scope.

The feature is **off by default**. With `ai_commander_enabled` false, nothing in `game/ai_commander/` runs, no file is written, no network request is made, and turn processing is byte-for-byte the stock behaviour.

1. What it is, and what it is not

It **is** a turn-level strategic advisor for RED. Once per RED turn it is given a sanitised briefing and returns a short structured document ranking priorities:

- an overall strategy posture,
- a ranked ordering of the active fronts,
- a per-front push posture (retreat / hold / probe / push / breakthrough),
- a ranked ordering of spending categories,
- a ranked ordering of known target sets with a mission purpose for each,
- a reserve policy, and
- a free-text `commander_intent` used only for the audit log and the UI.

It **is not** in the execution path. It never names a unit, a quantity, a price, a coordinate, a waypoint, a loadout, a flight size, a time on target, a damage result or a capture. Everything mechanical is still done by Retribution’s own deterministic code: the HTN planner, the procurement AI, the packages and flight plans, the mission generator, the combat and capture resolution, and the save file. The model’s output only changes the order in which the existing planner is offered its existing tasks, the stance set on a front, and the weighting the existing procurement code uses.

Concretely, the accepted decision is turned into a `CommanderDirective` (`game/ai_commander/directive.py`) which feeds three deterministic adapters in `game/ai_commander/execution.py` :

Adapter	Effect
<code>task_order_for(directive)</code>	Reorders the compound tasks offered to the existing HTN theater commander.
<code>apply_front_postures(...)</code>	Sets an existing <code>CombatStance</code> on a front, only if that stance is already legal there.
<code>DirectedProcurementAi</code>	Nudges the existing procurement split and re-serve share. Prices, availability and capacity checks stay in stock code.

If any of this is unavailable, refused, or fails, the built-in RED automation takes the turn instead and the campaign continues normally.

Two modes: COMMANDER and ACTIVE

The feature ships in two modes, selected by `ai_commander_mode` (`CommanderMode` in `enums.py`):

Mode	Requests/turn	What the model decides	Default
<code>commander</code>	1	Strategy only: the single ranked directive described above.	✓
<code>active</code>	up to 3 (6 worst case)	Everything COMMANDER decides, plus logistics (what to buy, repair, relocate, retask, transfer) and air tasking (which briefed targets to strike, with which owned airframes), applied through Retribution's own systems.	

Everything in sections 1 to 4 above is the COMMANDER contract and it holds in both modes unchanged. ACTIVE mode is strictly additive: it keeps the same strategy stage, then runs two further stages that make concrete, still player-legal orders. It never widens what the model may touch beyond what a human RED player could do from the same UI, it never sees more than the COMMANDER brief plus RED's own capability and operations views (section 5.5), and every order is re-checked against live state and executed only through existing game APIs (section 4.5). If any single stage produces nothing usable, that stage — and only that stage — falls back to the built-in staff; the turn never breaks (section 3.5).

The rest of this document describes the COMMANDER path first (sections 2–4), then the ACTIVE additions in the numbered subsections `x.5`. Choosing between the modes is covered in section 7.

2. Module map

All of it lives in `game/ai_commander/`, plus one call site and the UI.

Module	Responsibility
<code>enums.py</code>	Every closed vocabulary: intel policy, strategy, posture, target-set category, procurement category, mission purpose, reserve policy, confidence, precision, strength/affordability bands, personality, fallback reason.
<code>serialization.py</code>	<code>jsonable()</code> and <code>stable_hash()</code> — deterministic, ordering-independent JSON and hashing used for every hash in the brief and the audit record.
<code>intel.py</code>	<code>IntelProjector</code> and <code>RedCommanderBrief</code> . Builds the fair briefing; this is the fairness boundary .
<code>debrief.py</code>	<code>DebriefSummary</code> , <code>build_debrief_summary</code> , <code>classify_threat</code> and <code>ThreatCategory</code> . Turns the last mission's debriefing into a compact, RED-perspective after-action summary — RED's own losses attributed to their cause, plus RED's confirmed enemy kills — that <code>intel.py</code> folds into the brief. Also off the fairness boundary (see §4.6).
<code>postures.py</code>	Maps Retribution <code>CombatStance</code> values to/from the <code>FrontPosture</code> vocabulary, and computes which postures are legal on a front.
<code>decision.py</code>	The decision JSON schema, the example document, <code>extract_json_object</code> , and <code>validate_decision</code> — schema/ID/rank/enum validation.
<code>directive.py</code>	<code>CommanderDirective</code> , the validated, legality-checked intent object.
<code>legality.py</code>	<code>LegalityChecker</code> — rechecks an already schema-valid decision against live state (ownership, reachability, affordability, capacity, revision).
<code>pricing.py</code>	Token estimation, the <code>/models</code> catalogue, <code>ModelPrice</code> , and <code>CostLedger</code> (the per-turn cap).
<code>llmclient.py</code>	

Module	Responsibility
	The only networking code. Stdlib <code>urllib</code> only. Timeouts, bounded retries, usage parsing, redacting <code>__repr__</code> .
<code>prompt.py</code>	System prompt, personality text, user prompt, repair prompt, and the <code>response_format</code> request.
<code>execution.py</code>	The deterministic adapters listed above.
<code>audit.py</code>	<code>AiDecisionRecord</code> and <code>AuditLog</code> — the on-disk decision log. <code>StageRecord</code> and record schema <code>red-commander-audit/2</code> add the per-stage view for ACTIVE turns; v1 records still read.
<code>secretstore.py</code>	Where the API key lives, and how it is masked.
<code>config.py</code>	<code>AiCommanderConfig</code> , built from <code>Settings</code> plus the secret store. Carries <code>mode</code> ; <code>requests_per_turn</code> is 1 (COMMANDER) or 3 (ACTIVE).
<code>controller.py</code>	<code>RedCommanderTurn</code> — orchestrates one turn. Never raises. <code>_run_commander</code> is the one-call path; <code>_run_active</code> is the three-stage path.

ACTIVE mode adds these modules (all RED-only, all off the same fairness boundary), used only when `ai_commander_mode` is `active`:

Module	Responsibility
capabilities.py	CapabilityIndex — the RED-only catalogue of airframes, ground units, ships and doctrine RED owns or can buy, built straight from game data. It is what lets the model name a concrete airframe or unit without training-data guessing or reading BLUE's roster. CAPABILITY_CACHE memoises it per faction signature.
operations.py	OperationsProjector / OperationsBrief — RED's own bases, squadrons and observable targets, with generic BASE-N / SQN-N / TGT-N ids and never a coordinate. OperationsResolver maps those ids back to live objects for legality and execution.
plan.py	The LOGISTICS and AIR_TASKING JSON schemas, their order dataclasses, validate_logistics_plan / validate_air_tasking_plan (structural validation against the brief), and CommanderStage .
activeprompt.py	The per-stage system/user/repair prompts and response_format , assembled from the capability index, the operations brief and a summary of the prior stages.
planlegality.py	PlanLegalityChecker — the ACTIVE analogue of legality.py . Re-checks every schema-valid order against live state (budget across the stage, parking, supply source, transit route, runway eligibility, airframe ownership, revision) and returns ExecutableLogistics / ExecutableAirTasking plus Rejection s.
planexecution.py	PlanExecutor — applies the bound, legal orders through Retribution's own APIs only (AircraftPurchaseAdapter , GroundUnitPurchaseAdapter , begin_runway_repair , plan_relocation , set_auto_assignable_mission_types , TransferOrder , PackageFulfiller). One order's failure is caught and recorded; it never aborts the turn.

Outside the package:

- `game/coalition.py` — the single call site, on the RED coalition’s turn.
- `game/settings/settings.py` — the `ai_commander_*` options on a new “AI Opponent” settings page (`AI_OPPONENT_PAGE`), split into an “LLM Commander” and a “Fairness and Audit” section. `ai_commander_mode` chooses COMMANDER (default) or ACTIVE.
- `game/settings/textoption.py` — a new free-text option kind, needed for the model identifier and base URL.
- `qt_ui/windows/settings/aicommanderkey.py` — the API key widget. Writes to the secret store, never to `Settings` , so the key can never reach a save file.
- `qt_ui/windows/aicommander/QAiCommanderLogWindow.py` — the read-only decision log viewer, opened from the “AI Log” toolbar action in the main window.

3. Per-turn control flow

`RedCommanderTurn.run()` is the whole turn. It is written so that **it never raises**: every exit path either returns an accepted directive or a `CommanderTurnResult` carrying a `FallbackReason` , and the caller in `game/coalition.py` treats a fallback exactly like the stock code path.

1. **Disabled check.** If the feature is off, return immediately with `FallbackReason.DISABLED` . No brief is projected and no log file is written.
2. **Project the brief.** `IntelProjector(game, policy).project(...)` builds `RedCommanderBrief` from live state, with the fairness filter applied. This also produces `campaign_id_hash` (stable per campaign) and `campaign_revision` (a digest of turn number, RED budget, base ownership and RED deployable strength).
3. **Replay guard.** If the audit log already holds an accepted decision for this `campaign_revision` , that decision is replayed rather than re-bought. Re-entering a turn therefore cannot double-spend the budget or produce a different plan.
4. **Usability check.** A misconfigured setup (`is_usable` false — e.g. a remote base URL with no key) falls back with `FallbackReason.NOT_CONFIGURED` .
5. **Catalogue lookup.** One unbilled `GET /models` request records the live price, context length and notes for the configured model. If it fails, a deliberately pessimistic fallback price is used (\$3.00/\$15.00 per million in/out) and the record says the price is a fallback. Stale prices are never silently treated as authoritative.
6. **Seed the ledger.** `CostLedger(cap, already_spent=audit_log.spent_this_turn)` — the cap is enforced across the whole turn, not per call.
7. **Initial attempt.** Reserve the worst-case cost before sending. If the reservation would exceed the remaining cap, nothing is sent and the turn falls back with `FallbackReason.COST_CAP` .
8. **Validate.** `extract_json_object` then `validate_decision` . Fatal problems (non-object root, wrong `schema_version` , wrong `turn_id` , revision mismatch, unparseable enums) reject the whole document. Non-fatal problems (an unknown ID, a duplicate ID, a duplicate rank, ranks not starting at 1, an

illegal posture, an over-long intent, unexpected keys) drop or truncate just the offending element and record a `Rejection` for it.

9. **At most one repair attempt.** If and only if validation failed and the remaining budget covers another worst-case call, one repair request is sent containing the validation errors and the same legal IDs — no extra hidden state. There is no retry loop beyond this.
10. **Legality check.** `LegalityChecker` re-tests the surviving decision against live state: is that front still ours, is that base still reachable and owned, can we actually afford the cheapest item in that spending category, does the base have capacity, is the revision still current. Each refusal carries a human-readable reason and is logged.
11. **Accept or fall back.** If anything legal survives, the directive is built and handed to the deterministic adapters. If nothing legal survives, the turn falls back with `FallbackReason.NO_LEGAL_CONTENT`.
12. **Write the record.** One JSON file per attempt-set per turn. `AuditLog.write()` never raises; a failure to write returns `None` and the turn still completes.

Any transport error, HTTP error, timeout, unexpected exception, or repeatedly malformed output lands on the same deterministic fallback. The list of reasons is the `FallbackReason` enum: `DISABLED`, `NOT_CONFIGURED`, `COST_CAP`, `TRANSPORT_ERROR`, `HTTP_ERROR`, `TIMEOUT`, `MALFORMED_RESPONSE`, `STALE_RESPONSE`, `NO_LEGAL_CONTENT`, `UNEXPECTED_ERROR`.

3.5 ACTIVE mode: the three-stage turn

When `ai_commander_mode` is active, `run()` dispatches to `_run_active` instead of `_run_commander`. Steps 1-6 above are identical — same disabled check, same brief, same replay guard, same usability check, same catalogue lookup, and **the same single `CostLedger` seeded once for the whole turn.** What changes is that the turn now asks three questions in sequence, each its own request with its own schema, validator, legality pass and audit `StageRecord`:

1. **Operations projection.** Before stage 1, `_run_active` projects the two RED-only views the later stages plan against: the `OperationsBrief` (`OperationsProjector`) and the `CapabilityIndex` (`capability_index_for`). If either projection raises, the whole turn degrades to COMMANDER mode for that turn (`self._run_commander(...)`) with a note — it never breaks.
2. **Stage 1 — command intent.** `_active_command_stage` runs the exact COMMANDER decision (same schema, same validator, same `LegalityChecker`, section 4) and produces the same `CommanderDirective`. If stage 1 yields nothing legal, the entire turn falls back to the built-in RED automation, exactly as a COMMANDER turn would — the later stages are not attempted.
3. **Stage 2 — logistics.** `_active_logistics_stage` asks for a `LogisticsPlan` (aircraft buys, ground-unit buys, runway repairs, squadron relocations, re-taskings, ground transfers), validates it structurally against the brief (`validate_logistics_plan`), then re-checks every surviving order against live state with `PlanLegalityChecker.check_logistics`. What survives becomes an `ExecutableLogistics` and is applied immediately through `PlanExecutor` (real game APIs, section 4.5). An empty-but-well-formed plan is accepted (the commander legitimately ordered no logistics this turn); a plan that produced nothing legal, or a stale one, degrades **this stage only**.

4. **Stage 3 — air tasking.** `_active_air_tasking_stage` asks for an `AirTaskingPlan` (packages of flights against briefed targets), validates it structurally, re-checks each package and flight with `PlanLegalityChecker.check_air_tasking` (target resolves to a briefed TGO, airframe is one RED operates, mission is one that airframe can fly), and applies the survivors with `PlanExecutor.execute_air_tasking`, which builds real packages via `PackageFulfiller` and adds them to RED's ATO.
5. **Accept.** `record.execution_report` captures what was applied and what failed, and the turn is accepted carrying the stage-1 directive plus the execution report.

Per-stage degradation is the core robustness property. Each stage gets the same “initial request plus at most one repair” treatment as a COMMANDER turn (step 9 above), and the single turn-wide ledger is checked before every request, so the three stages together can never spend more than one cost cap. A stage that fails — malformed twice, no legal content, stale, or refused by the cap — is marked degraded with its `FallbackReason` and Retribution's built-in staff covers that domain for the turn, while every stage that already succeeded stays applied. Only a stage-1 failure loses the whole turn; a stage-2 or stage-3 failure never undoes an earlier stage and never breaks the turn. This is what `tests/ai_commander/test_active_turn.py` and the `active-*` dry-run scenarios exercise end to end.

4. Fairness and anti-cheat guarantees

The design goal is that the commander is interesting, not omniscient. Four separate mechanisms enforce that.

4.1 It only ever sees the projected brief

The model is given the `RedCommanderBrief` and nothing else. There is no code path that hands it a `Game`, a `ControlPoint`, a save file, a file path, an environment variable or a credential. The brief is an immutable dataclass tree of primitives and closed enums, produced by `IntelProjector`, and it is the only input to `prompt.build_messages`.

Everything RED can see about BLUE is deliberately coarse:

- Enemy strength on a front is a **band** (`_band_for_ratio`, thresholds at 0.4 / 0.8 / 1.25 / 2.5 of the friendly-to-enemy deployable ratio), not a count.
- Target sets are **classes with a count** (`TS-1`, “enemy air defences”, 3 known), with a `confidence` of `PROBABLE` and a `location_precision` of `AREA` — never a coordinate, never a unit name, never a group ID.
- Affordability of a spending category is a **band** (`NONE` / `LIMITED` / `COMFORTABLE`), computed against the cheapest legal item, so the model is never handed a price list to arbitrage.
- Anything BLUE that RED could not plausibly observe is dropped entirely before the brief is built: `_is_observable()` keeps an object only if it sits inside RED's own threat zone or within 120 km (`_REALISTIC_OBSERVATION_RANGE_METERS`) of a RED-held control point.

4.2 It can only speak the closed decision vocabulary

The decision schema has no free-numeric and no free-identifier field. Every ID the model returns must be one of the opaque, turn-scoped IDs that were in the brief (`FRONT-n` , `TS-n` , `PROC-n`); every enum value must be a member of the enum; ranks are integers that must be unique and start at 1; `commander_intent` is truncated at `MAX_INTENT_CHARACTERS` (600) and lists at `MAX_LIST_ENTRIES` (32). An invented ID is not a bug to be tolerated — it is a logged rejection.

Because the IDs are re-minted per turn from the projected brief, an ID cannot be used to smuggle knowledge from one turn to the next, and it cannot be used to address anything the brief did not already mention.

4.3 Legality is rechecked against live state, after validation

Schema validity is not authority. `LegalityChecker` runs against the live `Game` immediately before application and refuses, with a reason:

- a posture that is not legal on that front given its force balance (the message names the legal values, e.g. “legal values were retreat”),
- a spending priority RED cannot pay for — the message quotes the real number, e.g. “cheapest available airframe costs 22 but only 5 is available”, “cheapest ground unit costs 12 but only 5 is available”, “runway repair costs 100 but only 5 is available”,
- a front, base or target set that is no longer ours / reachable / present,
- a `campaign_revision` that no longer matches: “campaign state changed between briefing and application”.

If nothing legal survives, the refusal message says so explicitly — “nothing in the decision was legal against live state, so the built-in RED automation keeps control of this turn” — and the built-in automation runs.

4.4 Model output is untrusted data

The response is parsed as JSON and matched against the schema. It is never `eval 'd`, never used to build a path, never written into a save, and never used as a format string. Tool calls are refused in v1: if a response arrives with `tool_calls` , that is recorded as a `<tool_calls>` rejection. The save file is still written exclusively by Retribution’s own persistence code from its own objects.

4.5 ACTIVE mode keeps every guarantee, one legality rule per action

ACTIVE mode makes concrete orders, so it is where “player-equivalent” has to be proven, not asserted. The rule is simple: **the model may order only what a human RED player could do from the same screens, and every order is re-checked against live state and carried out through the same game code the player’s UI calls**. The four guarantees above hold unchanged, and stage 2/3 add a fifth: no order reaches the game except through `PlanExecutor` , which calls Retribution’s own APIs and never mutates state directly.

Each new action class is a player action with its own legality rule in `PlanLegalityChecker` and its own executor in `PlanExecutor` :

Action class (stage)	The player equivalent	Legality re-check against live state	Executed through
Aircraft purchase (2)	Buy aircraft into a squadron	Airframe is in the capability index; squadron exists and can base there; cumulative stage spend $\leq$ budget; parking available	<code>AircraftPurchaseAdapter.buy</code>
Ground-unit purchase (2)	Buy front-line ground units	Unit type in the index; a base with a supply source; cumulative spend $\leq$ budget	<code>GroundUnitPurchaseAdapter.buy</code>
Runway repair (2)	Repair a damaged runway	Base is RED's, runway actually damaged and repairable, repair cost $\leq$ budget	<code>ControlPoint.begin_runway_repair + adjust_budget</code>
Squadron relocation (2)	Move a squadron between bases	Destination is RED's, can operate the type, has parking	<code>Squadron.plan_relocation / cancel_relocation</code>
Squadron re-tasking (2)	Set a squadron's auto-assignable missions	Squadron is capable of each task requested	<code>Squadron.set_auto_assignable_mission_types</code>
Ground transfer (2)	Move ground units between bases	Units actually present at origin; a transit route exists via coalition.transfers.network_for()	<code>TransferOrder + coalition.transfers.new_transfer</code>
Air package + flights (3)	Plan a package against a target	Target resolves to a briefed TGO; airframe is one RED operates; mission is one that airframe can fly; package survives structural checks	<code>PackageFulfiller.plan_mission + coalition.ato.add_package</code>

Two anti-cheat consequences are worth spelling out because the tests and the dry run both prove them:

- **Invented targets collapse to nothing.** A package aimed at a target id that is not in the operations brief (the `active-cheat-target` scenario uses the sentinel `TGT-BLUE-SECRET-CANARY`) is dropped by structural validation before legality ever runs. The rejection is recorded (`packages[0].target_id: target identifier is not in the brief`), the stage is accepted with an empty air-tasking order, no package is added, and the built-in planner covers air tasking. The model cannot strike something it was never briefed on.
- **Unowned airframes collapse to nothing.** Flights flown by an airframe RED does not operate (the `active-cheat-airframe` scenario uses a BLUE-private airframe sentinel) are each refused (`packages[...].flights[...].aircraft_id: airframe is not one this faction operates`) and the package collapses. RED can only ever fly airframes in its own capability index.

The `campaign_revision` guard (a digest of turn, RED budget, ownership and own ground forces) rejects any plan whose live digest no longer matches, so a genuine external state change between briefing and application is thrown out. But in ACTIVE mode the commander's own applied stage-2 orders (a runway repair or a purchase debits the budget) change that digest, which would otherwise make its own stage-3 air tasking look stale. So once stage-2 logistics is applied, the guard is **re-baselined** to the digest that reflects the commander's own just-applied orders, and stage-3 is checked against that baseline instead of the turn-start one. The check is unchanged when no orders were applied (and in COMMANDER mode), and a real external change is still rejected — the re-baseline only absorbs the commander's own spending, never a third party's. `PlanLegalityChecker` takes an optional `expected_revision` override for this; the controller computes it with the same `IntelProjector` the checker uses so the two digests are always comparable.

Because the capability index and operations brief are built from **RED's own coalition only** — `CapabilityIndexBuilder` reads RED's owned and buyable units and never touches `coalition.opponent` or `game.blue`, and `OperationsProjector` projects only RED-owned control points and observable targets — the model is never even shown a BLUE unit type, squadron or hidden base to name. The intel-leak sentinels in section 5 cover the operations brief too.

4.6 The after-action debrief stays inside the fairness boundary

After every mission the brief carries a compact after-action summary of the previous turn so the commander can learn from its losses — the concrete motivation being reactions a human RED player would have: “my CAS was shot down by enemy fighters, so escort it next time”, or “my strike aircraft were downed by ship-launched SAMs, so kill the ships first”. That summary is built by `game/ai_commander/debrief.py` and folded into the brief by `intel.py`, and it is held to the same anti-cheat rule as everything else the model sees: **it reports only RED's own losses (attributed to their cause) and the enemy losses RED actually confirmed in combat** — never BLUE's budget, inventory, plans, or un-observed movements.

How it stays fair and cheap:

- **Where the data comes from.** DCS already emits an `S_EVENT_KILL` for each kill; the base plugin (`resources/plugins/base/dcs_retribution.lua`) now also records the cause of each kill (`target` , `by` , `by_type` , `weapon` , each captured defensively with `pcall`) into `kill_causes` in `state.json` . That is parsed backward-compatibly into `StateData.kill_causes` (`game/debriefing.py`); an older state file with no `kill_causes` simply yields an empty list.
- **RED perspective only.** In the debriefing, `player_*` is BLUE and `enemy_*` is RED. `build_debrief_summary` counts RED's losses from `loss_counts(Player.RED)` and RED's confirmed kills from `loss_counts(Player.BLUE)` (i.e. BLUE units RED was observed to destroy). It never reads BLUE's coalition, roster, or budget. The result is stored on the RED coalition as `Coalition.last_after_action` (a plain dict) by `MissionResultsProcessor.record_red_after_action` , the last step of `commit()` ; that step is wrapped in try/except so a debrief failure can never abort combat resolution.
- **Cause attribution.** Each RED loss with a known killer is classified by `classify_threat(unit_class)` into a small closed `ThreatCategory` vocabulary — enemy aircraft, ground SAM, naval (ship / ship-launched SAM), AAA, or ground fire. Auto-resolved combat (the `SimulationResults` path) carries no unit names or killers, so those losses cannot be attributed; they are counted from the loss totals and bucketed as `UNKNOWN` by remainder math (total minus what was attributed). Attribution is therefore best-effort and never over-claims.
- **Token cost is bounded.** `DebriefSummary.render_compact()` emits a short `[AFTER-ACTION ...]` block (a handful of lines, well under the brief's other sections) and returns `""` when the summary is empty, so a quiet turn adds nothing. `IntelProjector` rehydrates it from the stored dict via `DebriefSummary.from_dict` (garbage-tolerant) and renders it under `[AFTER-ACTION LAST MISSION]` , guarded so an empty summary is omitted.
- **Tested for leaks.** `tests/ai_commander/test_debrief.py` asserts the block never carries a BLUE sentinel (same approach as the intel-leak tests in §5), alongside classification, attribution, serialization round-trip, garbage tolerance and the render length bound.

5. What **REALISTIC** withholds versus **FULL_PARITY**

Two intel policies are selectable in the settings UI

(`ai_commander_intel_policy` , “Fairness and Audit” section).

This is a new restriction that Retribution itself does not have.

Stock Retribution has no campaign-layer fog of war. The human player's "Enemy Info" checkbox in the Intel window exposes the full enemy economy, aircraft inventory and ground forces, and the built-in OPFOR auto-planner reads live game state directly with no filtering at all. `REALISTIC` is therefore **not** a mirror of an existing Retribution feature and it does not re-implement one. It is a restriction invented for this feature, applied only to what the language model is shown, and it makes the LLM commander less informed than the built-in OPFOR auto-planner it replaces. The `opfor_autoplanner_aggressiveness` setting is a threat-radius ratio used by the stock planner; it is unrelated to this and is not a strategy dial.

The brief always carries a `withheld_fields` list naming what was removed, so an auditor can see the policy that was in force without diffing two runs.

Under `REALISTIC` (recommended, the default) nine field groups are withheld:

Withheld field	Meaning
<code>enemy_budget</code>	BLUE's cash on hand.
<code>enemy_income</code>	BLUE's income per turn.
<code>enemy_squadron_rosters</code>	BLUE squadron names, aircraft types and pilot counts.
<code>enemy_aircraft_inventory</code>	How many of what BLUE has, and where.
<code>enemy_exact_ground_unit_counts</code>	Replaced by a strength band per front.
<code>enemy_planned_flights</code>	BLUE's ATO / planned packages for the coming turn.
<code>enemy_unit_coordinates</code>	Exact positions; replaced by <code>AREA</code> precision.
<code>enemy_pending_purchases</code>	What BLUE has on order.
<code>campaign_save_data</code>	The save/ <code>Game</code> object itself, in any form.

Under `FULL_PARITY` only three remain withheld — `enemy_planned_flights`, `enemy_unit_coordinates` and `campaign_save_data`. `FULL_PARITY` deliberately matches what the human player can already read from the Intel window with "Enemy Info" ticked: exact enemy counts, budget and inventory. It exists so that a player who considers the Intel window part of normal play can give the AI the same picture. Under `FULL_PARITY` the projector also stops filtering by observability (`_is_observable` returns `True` unconditionally), upgrades confidence to `CONFIRMED` and precision to `PRECISE`, and populates `FrontView.enemy_unit_count`.

Even under `FULL_PARITY`, BLUE's planned flights, exact coordinates and the save data are still never sent, and the authority boundary of section 4 is unchanged — the model still cannot do anything except rank priorities.

The intel-leak tests in `tests/ai_commander/test_intel_leak.py` enforce this by seeding recognisable sentinel values on the BLUE side of a synthetic campaign (a canary budget, income, squadron name, aircraft name, pilot count, planned package name, unit counts, capacity, an undetected TGO and a base placed far outside RED's observation range) and then recursively scanning the fully serialised brief for any of them. They also assert that RED's own equivalents **are** present, so the filter cannot pass by being trivially empty.

5.5 The capability index and operations brief (ACTIVE mode)

COMMANDER mode ranks abstractions, so the intel brief is all it needs. ACTIVE mode names concrete airframes, units, bases and targets, so it is given two more RED-only views — and both are anti-cheat mechanisms, not just conveniences.

Capability index (`capabilities.py`). A catalogue of every airframe, ground unit, ship and doctrine value **RED already owns or is allowed to buy**, built by `CapabilityIndexBuilder` straight from RED's faction and coalition data. Each entry carries the facts the model needs to plan — price, the mission roles the airframe is `capable_of`, year introduced, a few combat characteristics — and nothing else. It exists for two reasons:

- Anti-hallucination. Without it, a model would name airframes and units from its training data ("send the Su-57s") that this faction may not field in this era. The index is the closed vocabulary of buyable/ownable things, so `validate_logistics_plan` / `validate_air_tasking_plan` can reject anything outside it as a structural error before legality even runs.
- Anti-cheat. It is built from RED's side only — it never reads `coalition.opponent` or `game.blue` — so the model is physically never shown a BLUE airframe or unit type to copy. That is what makes the `active-cheat-airframe` refusal a structural certainty, not a filter that might be forgotten.

It is derived from live data every turn (memoised in `CAPABILITY_CACHE` by a `sha256` of the faction's unit data, so an unchanged faction is not rebuilt), and its `content_hash()` is recorded in the audit log so a reader can tell which catalogue a turn planned against.

Operations brief (`operations.py`). RED's own operational picture: its bases, its squadrons, and the targets it can observe, each with a generic `BASE-N` / `SQN-N` / `TGT-N` identifier and never a coordinate (a target's location is given only as `near=BASE-n`). It runs through the same `IntelPolicy` and observability filter as the intel brief, so it withholds the same BLUE-private fields (section 5) and only surfaces targets inside `OBSERVATION_RANGE_METERS`. The generic ids are the plan vocabulary: the model plans in `TGT-2` / `SQN-1`, and `OperationsResolver` maps those back to the live game objects for legality checking and execution, so the model never handles a real object, a save reference or a coordinate. Its `content_hash()` is recorded too.

6. Auditing a turn

Every turn that reaches the projector writes one JSON file. Nothing is silently dropped: refusals, fallbacks and cost are all in the record.

Where. `<audit root>/AiDecisions/<campaign_id_hash>/turn_<NNNN>_<II>.json`, where `<NNNN>` is the turn number and `<II>` distinguishes multiple records for the same turn (capped at `MAX_RECORDS_PER_TURN = 50`). The audit root defaults to the user data directory (`~/local/share/DCSRetribution` on POSIX, `%LOCALAPPDATA%\DCSRetribution` on Windows) and can be overridden. `campaign_id_hash` is a hash of theater, faction names and base names — it contains no path and no personal data.

Reading it. In the GUI, use the **AI Log** toolbar action in the main window; it lists the records for the current campaign and shows one decision at a time, read-only. On disk the file is plain JSON, so `jq` works fine.

To reconstruct a turn, walk the record in this order:

1. **Identity** — `record_schema_version`, `written_at`, `campaign_id_hash`, `turn_id`, `campaign_revision`, `personality`.
2. **What the commander knew** — `intel_policy`, `intel_hash`, `intel_brief` (the entire brief as sent, so fairness is auditable after the fact), `intel_rendered` (the compact text actually placed in the prompt), and `intel_brief.withheld_fields`. If you want to prove no BLUE-private value was ever sent, grep this object; it is the complete input.
3. **What it was asked** — `decision_schema_hash`, plus `attempts[*].prompt_messages` when raw prompt logging is enabled (`prompt_logging_enabled` records whether it was). When it is off, `attempts[*].prompt_hash` still lets you prove which prompt was sent without storing the campaign detail it contains.
4. **The provider round trips** — `base_url`, `configured_model`, `catalog_retrieved_at`, `catalog_input_price_per_million`, `catalog_output_price_per_million`, `catalog_context_length`, `catalog_notes`, then per attempt: `attempt`, `kind` (`initial` / `repair`), `started_at`, `latency_seconds`, `requested_model`, `actual_model` (so provider-side alias resolution is visible), `response_id`, `provider`, `finish_reason`, `retries`, `http_status`, `error`, the token counts (`prompt_tokens`, `completion_tokens`, `cached_tokens`, `reasoning_tokens`, `total_tokens`), `reserved_cost`, `actual_cost`, `cost_is_estimated`, and `response_text` / `response_hash`.
5. **What it proposed** — `parsed_decision` (schema-valid, before legality).
6. **What was refused** — `rejections`, a list of `{element, reason, value}` objects. This is the interesting part when the AI “did nothing”: every dropped element is here with its reason.
7. **What was accepted** — `accepted_directive`.
8. **What deterministic code then did** — `planner_task_order` (the order the HTN planner was offered its tasks), `posture_applications` (one entry per requested posture, applied or not, with why), `procurement_notes`.
9. **Cost** — `cost_cap_per_turn`, `prior_cost_this_turn`, `estimated_cost`, `reserved_cost`, `actual_cost`, and the derived `total_prompt_tokens` /

`total_completion_tokens` . `cost_is_estimated` false means the figure came from the provider's own `usage` block and is authoritative.

10. **Outcome** — `accepted` , `fallback_reason` , `fallback_policy` , `replayed_from_turn_record` , `notes` .

Useful checks:

```
# Which turns fell back, and why?
jq -r '[".turn_id, (.fallback_reason // "accepted")] | @tsv' \
  ~/.local/share/DCSRetribution/AiDecisions/*/turn_*.json

# Everything that was refused this campaign, with reasons.
jq -r '.rejections[] | [.element, .reason] | @tsv' \
  ~/.local/share/DCSRetribution/AiDecisions/*/turn_*.json

# Prove the brief carried no BLUE budget/inventory (REALISTIC).
jq '.intel_brief.withheld_fields' \
  ~/.local/share/DCSRetribution/AiDecisions/*/turn_0007_00.json

# What did a turn actually cost?
jq '{cap: .cost_cap_per_turn, actual: .actual_cost, estimated: .cost_is_estimated}' \
  ~/.local/share/DCSRetribution/AiDecisions/*/turn_0007_00.json
```

The API key never appears in any of this. It is not in the record, not in the config's `to_dict()` (which emits only a boolean `api_key_configured`), and not in any log line — `ChatCompletionClient.__repr__` prints `api_key=<redacted: set>` and `AiCommanderConfig.describe()` masks the value.

7. Provider and model configuration

All of the following is on the **AI Opponent** settings page.

Setting	Default	Notes
<code>ai_commander_enabled</code>	<code>False</code>	Master switch. Off means the feature does not exist at runtime.
<code>ai_commander_model</code>	<code>deepseek/deepseek-v4-flash-0731</code>	The provider's exact model ID. Prefer a dated ID over a <code>latest</code> alias for reproducibility.
<code>ai_commander_base_url</code>	<code>https://openrouter.ai/api/v1</code>	Any OpenAI-compatible chat-completions endpoint.
<code>ai_commander_personality</code>	<code>balanced</code>	<code>cautious</code> / <code>balanced</code> / <code>aggressive</code> / <code>attritional</code> . Changes only how the commander is briefed, never what it is allowed to do.
<code>ai_commander_intel_policy</code>	<code>realistic</code>	See section 5.
<code>ai_commander_cost_cap_per_turn</code>	<code>0.5</code>	US dollars, per RED turn, worst case, enforced before sending.
<code>ai_commander_timeout_seconds</code>	<code>90</code>	Exceeded means fall back.
<code>ai_commander_max_output_tokens</code>	<code>12000</code>	Caps the size, and therefore the cost, of each response.
<code>ai_commander_log_prompts</code>	<code>True</code>	Store raw prompts in the audit record. They contain campaign information (though no BLUE-private data), so this is user-configurable.
<code>ai_commander_fallback_to_builtin</code>	<code>True</code>	Keep this on.

Where the key lives

The API key is **not** a setting and is **not** part of the campaign. It is read, in order:

1. the `OPENROUTER_API_KEY` environment variable, if set; otherwise
2. `ai_commander_secrets.json` in the user data directory
(`~/.local/share/DCSRetribution/` on POSIX,

`%LOCALAPPDATA%\DCSRetribution\` on Windows), under the key `openrouter_api_key`.

The file is created with mode `0600` and unrelated fields in it are preserved on save. Because the key lives outside `Settings`, it can never be pickled into a `.retribution` save, shared with a save file, or committed. It is also never logged: `secretstore.mask()` renders a short key as all asterisks and a long one as `abcdef...wxyz` (N chars), and every `__repr__` on the path is redacting.

OpenRouter

Point `ai_commander_base_url` at `https://openrouter.ai/api/v1` (the default), set a model ID, and paste the key into the API key box on the AI Opponent settings page — or export it:

```
export OPENROUTER_API_KEY=<PLACEHOLDER>
```

The client sends `Authorization: Bearer <key>`, requests an explicit `model` (never the account default), asks for a JSON response format, caps output tokens, and does not enable any provider plugins. HTTP 429 and 5xx get a small bounded backoff; 401 and 402 do not retry — they fall back immediately, since a bad key or an empty balance will not fix itself within a turn.

Reasoning models are supported. Reasoning models (e.g. the shipped `deepseek/deepseek-v4-flash-0731`) emit their thinking into a separate channel before the visible answer. When `content` comes back empty, the client falls back to reading the reasoning channel (`reasoning_content`, then `reasoning`, each in plain-string or content-parts form) so the JSON answer is still recovered — this is why the previous empty-`content` `malformed_response` fallbacks no longer occur. As defence in depth, on structured decision calls (those that carry a `response_format`) the client also passes OpenRouter's `reasoning.max_tokens` control, capped at `min(2000, max_output_tokens // 2)`, so the model always leaves budget for the actual answer instead of spending it all on hidden reasoning. The field is ignored by providers that do not understand it (plain OpenAI, Ollama), so it is a safe no-op there.

Local models (Ollama and friends)

Any OpenAI-compatible server works. For Ollama:

```
ai_commander_base_url = http://localhost:11434/v1
ai_commander_model    = <whatever you have pulled>
```

`AiCommanderConfig.is_local` recognises loopback/local base URLs; a local endpoint needs no API key (`requires_api_key` is false) and has no per-turn cost. The `/models` catalogue lookup is still attempted, and if it is missing or shaped differently the pessimistic fallback price is used for reservation arithmetic only — with a local model the cap effectively never binds. Local models are the right choice for anyone who does not want a paid dependency; the only requirement is that the model can return a JSON object, and a malformed reply just means the built-in automation takes the turn.

The cost cap

`CostLedger` (in `pricing.py`) enforces

```
spent_this_turn + reserved_in_flight + worst_case_next_call <= cost_cap_per_turn
```

Worst case is computed before the request from the measured prompt length and the configured output cap, using the live catalogue price (or the pessimistic fallback). If the reservation does not fit, **the request is never sent** and the turn falls back with `FallbackReason.COST_CAP`. When a response arrives, the reservation is released and the actual charge settled — from the provider's own `usage.cost` when present (recorded with `cost_is_estimated` false), otherwise estimated from catalogue prices and returned token counts. Caching is treated as a saving that may or may not materialise; it is never assumed when enforcing the cap.

8. What a turn costs

There is no provider tokeniser in this repository and this feature adds **zero runtime dependencies**, so token counts come from `pricing.estimate_tokens()` — `ceil(len(text) / 4.0 * 1.15)`, the same estimator the controller uses to size its reservation. The 1.15 factor deliberately over-counts so the reservation errs high. The character counts below are exact measurements of the real assembled prompt taken by `tools/ai_commander_dryrun.py`; the token counts are that estimator applied to them. A provider that reports real usage overrides the estimate for billing.

Measured prompt sizes on the harness's synthetic campaign:

Text	Characters	Est. tokens
initial prompt (cautious)	7230	2079
initial prompt (balanced)	7169	2062
initial prompt (aggressive)	7203	2071
initial prompt (attritional)	7184	2066
repair prompt (balanced)	7449	2142
representative response	636	183

How the briefing grows with campaign size (measured by widening the synthetic theater and reassembling the prompt each time):

Fronts	Bases	Target sets	Characters	Est. tokens
1	4	6	7169	2062
2	6	6	7327	2107
4	10	6	7645	2198
8	18	6	8277	2380
16	34	6	9585	2756
32	66	6	12193	3506

The brief is a fixed-shape summary — one line per front, one per known target class, one per procurement option — so it scales with the number of fronts, not with the number of units, aircraft or objects in the campaign. A 32-front theater is far larger than any stock campaign.

Call-count assumption. At most **two** billed chat completions per RED turn: the initial request, plus exactly one repair request if and only if the first response fails validation. There is no retry loop beyond that. One unbilled `GET /models` lookup happens per turn. Provider-side transport retries (429/5xx) re-send the same request and are billed only if a response is served. If a future build adds a second decision point per turn, double the figures below.

- typical turn = 1 call: 2079 in + 183 out
- worst-case turn = 2 calls: 4221 in + 4000 out (both replies at the 2000-token cap)

Model	\$/M in	\$/M out	Typical turn	Worst-case turn	Headroom vs \$0.50
deepseek/ deepseek-v4- flash-0731 (shipped de- fault)	0.09	0.18	\$0.00022	\$0.00110	455x
qwen/ qwen3.7- flash (cheapest surveyed)	0.03	0.13	\$0.00009	\$0.00065	773x
openai/ gpt-5.6-luna	0.10	0.60	\$0.00032	\$0.00282	177x
z-ai/glm-5.2	0.76	2.42	\$0.00202	\$0.01289	39x
moonshotai/ kimi-k3	3.00	15.00	\$0.00898	\$0.07266	7x
catalogue unavailable (built-in pessimistic price)	3.00	15.00	\$0.00898	\$0.07266	7x

Verdict: every surveyed model's worst-case turn stays under the \$0.50 ceiling, including the pessimistic fallback price used when the catalogue cannot be read. Even the most expensive model surveyed leaves roughly a 7x margin, so the cap is a safety net against a pathological prompt or a price change, not a routine constraint. A hundred-turn campaign on the shipped default costs on the order of two US cents.

Note on the token cap. The worst-case figures above use the harness's 2000-token reference cap. The shipped `ai_commander_max_output_tokens` default was raised to 12000 so reasoning-heavy models no longer exhaust the budget on hidden thinking and return empty stages. At 12000 the COMMANDER worst-case output reservation is six times larger, but even the most expensive surveyed model (`kimi-k3` , and the identical pessimistic fallback price) reaches only about \$0.37 per turn — still under the \$0.50 ceiling for every surveyed model.

These are estimates from measured character counts and published prices, not observed invoices. Prices were recorded on 2026-08-05 and change without notice; the controller always prices from the live `/models` catalogue at runtime and records the price it used in the audit record.

8.1 ACTIVE mode: three stages, up to six calls

ACTIVE mode replaces the single strategic decision with **three sequential stages per RED turn** — `command` , `logistics` , `air_tasking` . Each stage is one billed completion; each may be repaired once, so the hard ceiling is **six billed completions per turn** (three initial + three repairs). The stages share one `CostLedger` seeded from `audit_log.spent_this_turn` , so the cap is applied across the whole turn, not per stage — a stage is refused before sending if its worst-case reserve would push the turn total past the cap.

The figures below are exact measurements from

`tools/ai_commander_dryrun.py --scenario active-valid` , which assembles and prices the three real stage prompts against the synthetic campaign. Each later stage carries a short summary of the prior stages, so `logistics` and `air_tasking` prompts are slightly larger than `command` .

Stage	Est. tokens in	Est. tokens out
<code>command</code>	2688	183
<code>logistics</code>	3225	70
<code>air_tasking</code>	2965	143
turn total (3 calls, no re-pair)	8878	396

- typical turn = 3 calls: 8878 in + 396 out
- worst-case turn = 6 calls: 17761 in + 12000 out (every stage repaired once, all six replies at the 2000-token cap)

Model	\$/M in	\$/M out	Typical turn	Worst-case turn	Headroom vs \$0.50
deepseek/ deepseek-v4- flash-0731 (shipped de- fault)	0.09	0.18	\$0.00087	\$0.00376	133x
qwen/ qwen3.7- flash (cheapest surveyed)	0.03	0.13	\$0.00032	\$0.00209	239x
openai/ gpt-5.6-luna	0.10	0.60	\$0.00113	\$0.00898	56x
z-ai/glm-5.2	0.76	2.42	\$0.00771	\$0.04254	12x
moonshotai/ kimi-k3	3.00	15.00	\$0.03257	\$0.23328	2x
catalogue unavailable (built-in pessimistic price)	3.00	15.00	\$0.03257	\$0.23328	2x

Verdict: even in ACTIVE mode, with all three stages repaired and every reply at the token cap, every surveyed model's worst-case turn stays under the \$0.50 ceiling — including the pessimistic fallback price used when the catalogue cannot be read, which lands at roughly \$0.23, a little over 2x headroom. The shipped default keeps two orders of magnitude of margin. ACTIVE mode costs roughly four times a COMMANDER turn because it makes three to six calls instead of one to two, and the cap absorbs that comfortably.

Note on the token cap. As with COMMANDER mode, the worst-case figures above use the harness's 2000-token reference cap. With the shipped `ai_commander_max_output_tokens` default now 12000, the worst-case output reservation for a fully-repaired six-call turn is six times larger. The shipped default and the cheaper models stay well under \$0.50 even then (`deepseek` $\approx$ \$0.015), but the two most expensive surveyed models (`kimi-k3` and the identical pessimistic fallback price) would reserve about \$1.13 for that worst-case turn — over the ceiling. Because the shared `CostLedger` reserves each stage's worst case before sending, it refuses those later stages and they fall back to the built-in planner, so the \$0.50 ceiling is still never exceeded — the cap doing exactly its job. Operators who select such an expensive model in ACTIVE mode should lower `ai_commander_max_output_tokens` to keep all three stages within the cap.

9. Tests and the dry-run harness

Unit tests

`tests/ai_commander/` follows the existing pytest conventions (no `conftest.py`, `__init__.py` per package, `MagicMock(spec=...)` / `cast("Game", ...)` stubs, fully annotated). `tests/ai_commander/fakes.py` builds a synthetic RED-vs-BLUE campaign entirely out of stubs — no DCS, no mission files, no network:

File	Covers
<code>test_intel_leak.py</code>	The fairness boundary. BLUE sentinel values must not appear anywhere in the serialised brief under <code>REALISTIC</code> ; RED's own equivalents must appear.
<code>test_decision_schema.py</code>	Malformed JSON, empty bodies, prose-wrapped JSON, missing/wrong <code>schema_version</code> and <code>turn_id</code> , unknown IDs, duplicate IDs, duplicate ranks, ranks not starting at 1, unknown enum values, unexpected keys, over-long intent, list-length caps.
<code>test_legality.py</code>	Overspending, fronts and bases RED does not own or cannot reach, buying aircraft with no squadron to fly them, bases with no supply source or unrepairable runway, illegal postures, and stale revisions — each rejected with a logged reason rather than applied. Also asserts the directive carries no amounts, so quantity and base-capacity limits stay with the engine.
<code>test_cost_cap.py</code>	Ledger arithmetic, refusal before sending when the worst-case reserve exceeds the cap, reserve/release/settle, and that the controller falls back instead of raising.
<code>test_fallback.py</code>	Transport error, HTTP error, timeout, repeatedly malformed output, unexpected exception — every one ends in the deterministic fallback so a turn never breaks.
<code>test_secrets_and_audit.py</code>	Key masking and redaction, that the key is absent from configs/records/reprs, audit record shape, per-turn cost accounting, and the replay guard.
<code>test_debrief.py</code>	The after-action summary (§4.6): <code>classify_threat</code> mapping, cause attribution and the <code>UNKNOWN</code> remainder bucket, RED-perspective loss/kill counting, serialization round-trip and garbage tolerance, the <code>render_compact</code> length bound, and — same as <code>test_intel_leak.py</code> — that the rendered block leaks no BLUE information.

The ACTIVE-mode suite adds:

File	Covers
<code>test_capabilities.py</code>	The capability index: only airframes this faction actually operates are listed, only reachable/owned bases and fronts appear, procurement options match the faction, and BLUE capabilities never leak in. Index correctness is what makes an ACTIVE order legal-or-not decidable.
<code>test_operations_brief.py</code>	The richer ACTIVE brief. Same <code>REALISTIC</code> fairness boundary as <code>test_intel_leak.py</code> but for the extra ACTIVE fields, plus per-stage intel-leak sentinels — no BLUE budget, inventory, squadron or base detail in any stage prompt.
<code>test_active_plan.py</code>	The ACTIVE decision schema for all three stages: malformed/empty/prose-wrapped JSON, wrong <code>schema_version / turn_id</code> , unknown/duplicate IDs, unknown enums, unexpected keys, list-length caps, per-stage shape.
<code>test_active_legality.py</code>	One legality rule per new action class — packages, procurement, runway repair, aircraft transfer, relocation, posture — each illegal case rejected with a logged reason: targets/airframes/bases not in the brief, aircraft the faction does not operate, unreachable or unowned bases, and stale revisions.
<code>test_active_execution.py</code>	The execution adapters translate an accepted directive into existing planner inputs (package fulfilment, procurement, runway repair, transfer, relocation) and never mutate state directly; a rejected order produces no side effect.
<code>test_active_turn.py</code>	The full three-stage turn: multi-call cost accounting across one shared ledger, per-stage fallback (any stage failing falls back deterministically without breaking the turn), and the cap refusing a later stage when the turn budget is exhausted.

CLI dry-run harness

`tools/ai_commander_dryrun.py` runs a complete offline RED turn against the synthetic campaign and reports what happened. It works with **no API key** and exits 0.

```
python tools/ai_commander_dryrun.py          # everything
python -m tools.ai_commander_dryrun --no-live # skip the optional live call
python tools/ai_commander_dryrun.py --list
python tools/ai_commander_dryrun.py --scenario cheating-intel --verbose
python tools/ai_commander_dryrun.py --json /tmp/dryrun.json
```

It prints: the synthetic campaign layout; an intel-fairness sentinel scan; a scenario table; a field-by-field dump of a real audit file; the token measurements and the cost table in section 8; and a summary. The scenarios are

```
valid , repair , malformed , truncated-json , cheating-intel ,
cheating-overspend , illegal-posture , stale-state , tool-calls ,
transport-failure , timeout , catalogue-unavailable , cost-cap ,
not-configured , disabled .
```

ACTIVE mode adds six three-stage scenarios: `active-valid` (a clean command/logistics/air-tasking turn), `active-cheat-target` (an air-tasking order against a target not in the brief is rejected), `active-cheat-airframe` (procurement of an airframe the faction does not operate is rejected), `active-malformed-logistics` (one stage returns junk, is repaired, the turn continues), `active-tasking-fails` (air tasking stays malformed and falls back without breaking the turn), and `active-cost-cap` (a tight cap refuses the later stages once the turn budget is spent).

Each scenario asserts an expected outcome, so the harness doubles as an integration smoke test. When run with no arguments it also prints a measured ACTIVE-mode per-stage token and per-turn cost table against the \$0.50 cap (section 8.1). If `OPENROUTER_API_KEY` is set it will additionally make **one** real request to confirm the provider path end to end; without a key it says so and skips it. The key is never printed — only whether one was found.

10. Deferred scope

ACTIVE mode delivers part of what the original COMMANDER version deferred: the granular role now exists for **air tasking and logistics**. In ACTIVE mode the model shapes individual strike/CAP/CAS/DEAD packages (which targets, which airframes, which base) and directs the turn's procurement, runway repair, aircraft transfers and squadron relocation — all through the same fairness, legality, cost and audit machinery as COMMANDER mode.

The following are still deliberately **not** in this version:

- **Naval warfare.** Enemy shipping appears as a target class the commander can rank, but there is no naval-specific decision, no carrier group tasking and no sea-control reasoning.

- **Carrier operations.** No decisions about carrier station, recovery windows or air-wing composition.
- **Transports and cargo logistics.** Convoys and cargo ships are visible as target classes; RED's own transport, cargo and airlift planning is untouched and stays with the existing deterministic code. ACTIVE-mode logistics covers procurement, runway repair, transfer and relocation only.
- **BLUE as an AI commander.** The hook in `game/coalition.py` fires only for RED. Nothing in the package assumes RED, but running it for BLUE would need a fairness review of its own (the player's own information would become the model's), a second cost budget, and UI work.
- **Flight-plan and real-time detail.** Even in ACTIVE mode the model chooses the package's target, airframe and base; it does not author individual waypoints, formations, timing or in-mission unit control. Flight planning stays with the existing deterministic planner.

How the interfaces were shaped to grow into the active role

The v1 COMMANDER boundary was authority, not granularity, and the seams were left where the active role would need them. ACTIVE mode was built into exactly those seams — which is why it needed no change to the fairness filter, the cost cap semantics or the audit record format:

- **The brief is the only input.** A finer role adds fields to `RedCommanderBrief` and entries to `withheld_fields`; it does not change what the model is allowed to touch. `intel_hash` already versions the brief so a richer brief is distinguishable in the audit log.
- **IntelProjector is the single filter.** Read-only planning tools for a future tool-calling role would be built on top of it, so they can only ever expose the already-sanitised RED view. `_is_observable()` is the one place observability is decided.
- **The decision schema is generated from the brief.** `decision_json_schema(brief)` and `decision_schema_hash(brief)` are derived, not hard-coded, so adding a decision block (for example package-level intent) does not require rewriting the validator, and existing audit records stay interpretable via their recorded schema hash.
- **LegalityChecker is the choke point.** Any new decision block gets a new legality rule and a new `Rejection` reason. The contract — schema validity is not authority; live state decides — does not change.
- **CostLedger is per turn, not per call.** It is seeded from `audit_log.spent_this_turn`, so a role that makes several calls (or per-tool authorisation with call-count limits and cumulative turn budgeting) plugs into the existing reserve/release/settle cycle without touching the cap semantics.
- **LlmAttempt records are a list.** `AiDecisionRecord.attempts` already holds many attempts with per-attempt cost and token accounting, so a multi-call role needs no record-format change.
- **The execution adapters are additive.** `execution.py` converts a directive into existing planner inputs. A finer role adds another adapter next to `task_order_for` / `apply_front_postures` / `DirectedProcurementAi`; it does not need a new execution path.

Real-time DCS unit control, mission scripting, arbitrary coordinates and any direct mutation of persistence stay outside the LLM boundary until they are separately designed and tested.

11. What has not been validated here

This feature was developed and tested on Linux. DCS World and the packaged Retribution build are Windows-only, so the following is written and reviewed but **not** exercised:

- a real campaign turn end to end inside DCS, including mission generation,
- the packaged (PyInstaller) build,
- Qt rendering of the new settings page, the key widget and the AI Log window,
- save/load round-tripping of a campaign created with the new settings present,
- real provider latency, real invoices and real `usage` blocks from OpenRouter.

ACTIVE mode adds these to the list. The stages, schema, legality, cost accounting and fallback are all exercised headlessly, but the execution adapters are driven against no-op fakes in the dry-run harness (`_NoopAircraftAdapter`, `_NoopGroundAdapter`, `_FakeFulfiller`), so what remains unexercised is the adapters applied to a **real theater**:

- an accepted air-tasking directive actually fulfilled by the real `PackageFulfiller` into flyable packages on a live campaign,
- directed procurement actually spending against RED's real purchase adapter,
- runway repair, aircraft transfer and squadron relocation actually applied to a real campaign save and surviving a save/load round-trip.

Everything in sections 3 to 9 is covered by the headless tests and the dry-run harness, which do not need DCS.